

UNIVERSIDAD NACIONAL

Sede Regional Brunca

EIF411 — Desarrollo y Diseño de Plataformas Móviles

Laboratorio 3: API + Sincronización

Proyecto: PlantasMon

Profesor: Daniel Granados Murillo

Fecha de entrega: 24/04/2026

1. Módulos Generales de la App

PlantasMon es una aplicación móvil de identificación y seguimiento de plantas con perfiles gamificados, desarrollada con Expo SDK 54, TypeScript, Firebase y Expo Router. Los módulos principales identificados son:

Módulo	Funcionalidad	Ruta / Ubicación
Auth	Login, registro, cierre de sesión con Firebase Auth	app/(auth)/
Home / Dashboard	Vista principal con resumen de plantas y estado de conexión	app/(tabs)/index
Colección de Plantas	Listado de plantas del usuario, acceso a detalle	app/(tabs)/plants
Detalle / Editar Planta	Ver y editar información de una planta específica	app/plant/[id]
Identificación con IA	Captura de foto + llamada a PlantNet API para identificar la planta	app/(tabs)/identify
Cámara / Captura	Uso de cámara nativa con expo-camera para tomar foto de la planta	components/CameraView
Perfil de Usuario	Datos del usuario, estadísticas, nivel,	app/(tabs)/profile

	plantas compañeras	
Plantas Compañeras	Sistema de compatibilidad entre plantas, datos estáticos	components/CompanionPlants
Recordatorios / Cuidados	Notificaciones locales de riego y cuidado	app/(tabs)/reminders

1.2 Módulos con Soporte Offline

Se identificaron los módulos según su capacidad de funcionar sin conexión a internet:

Módulo	Tipo de operación	Soporte offline	Justificación
Colección de Plantas	LECTURA	Sí	Se cachea en AsyncStorage al cargar; disponible para consulta sin red
Detalle / Editar Planta	ESCRITURA	Sí (sync)	Cambios se guardan en cola local y se sincronizan al volver la conexión
Planta Compañera	LECTURA/ESCRITURA	Sí	Es la planta favorita del usuario — la más visible en home y perfil. Debe estar siempre disponible; cambiarla o editarla se sincroniza después
Recordatorios / Calendario	LOCAL	Sí	Vista de calendario interno con cuidados por planta; notificaciones locales con expo-notifications. Sin dependencia de red
Home / Dashboard	LECTURA	Parcial	Muestra datos del cache local; no se actualizan hasta recuperar la red
Perfil de Usuario	LECTURA	Parcial	Datos básicos disponibles en cache; ediciones quedan pendientes de sync
Cámara + Identificación con IA	API EXTERNA	No	Flujo 100% online: foto → PlantNet → resultado → guardar. Sin red se bloquea desde el inicio con mensaje al usuario
Auth — Login/Registro	LECTURA/ESCRITURA	No	Validación en Firebase Auth requiere conexión. Si ya inició sesión, el token se mantiene localmente

1.3 Justificación de Módulos Offline

El criterio para priorizar un módulo como offline fue si su ausencia interrumpe el uso cotidiano de la app. PlantasMon es una herramienta de seguimiento personal: el usuario interactúa con sus plantas diariamente, no solo cuando tiene señal.

Módulos priorizados para offline:

- Colección y detalle de plantas: son el núcleo de la app. El usuario necesita poder ver y editar sus plantas en cualquier momento. Los cambios hechos sin red se guardan en cola y se sincronizan cuando vuelve la conexión.
- Planta compañera: es la favorita entre todas las plantas del usuario — la más destacada, la que aparece en home y perfil. Al ser el elemento de mayor visibilidad, debe estar disponible offline sin excepción. Cambiarla o editarla genera una operación pendiente que se sincroniza después. Internamente se maneja como un campo `companionPlantId` en el perfil del usuario.
- Recordatorios y calendario: los cuidados de cada planta (riego, fertilización, etc.) se gestionan en un calendario interno de la app. Las notificaciones se programan con expo-notifications de forma local, sin depender de ningún servidor. El usuario puede consultar y editar su calendario aunque esté sin señal.

Módulos que requieren conexión obligatoria:

- Cámara + Identificación con IA: el flujo de agregar una planta es completamente online — se toma la foto, se manda a PlantNet, se obtiene el resultado y se guarda en Firestore, todo en un paso. Si no hay conexión, se muestra un mensaje claro y no se permite avanzar. No se implementa encolamiento de fotos porque sin la identificación el resultado no tiene utilidad para el usuario.
- Login / Auth: la autenticación inicial requiere Firebase Auth. Una vez que el usuario inició sesión, Firebase mantiene el token localmente, por lo que no se le pide volver a autenticarse si ya tenía sesión activa.

2. Mensajes y Contenido Pendiente

Cuando la app opera sin conexión, el mayor riesgo no es técnico sino de confianza: el usuario necesita saber que su trabajo está guardado y que se enviará cuando la red vuelva.

2.1 Estrategias para Mostrar Contenido No Sincronizado

La estrategia adoptada para PlantasMon combina un banner de estado persistente con toasts de transición, que cubre los dos momentos más críticos de la experiencia offline.

Banner de estado offline

El banner se ubica fijo entre el header de la app y el contenido principal. Su función es acompañar al usuario durante toda la sesión sin conexión sin interrumpirla. Tiene tres estados:

- Estado offline (color amber): aparece cuando NetInfo detecta pérdida de conexión. Muestra texto como "Sin conexión · N cambios pendientes".

- Estado sincronizando (color azul + barra de progreso): reemplaza al anterior cuando la conexión regresa y Firestore empieza a procesar la cola.
- Estado de error (color rojo + botón reintentar): aparece si uno o más elementos no pudieron sincronizarse.

El banner desaparece únicamente cuando la conexión está activa y no hay operaciones pendientes.

Toasts de transición

Los toasts cubren los instantes exactos en que algo cambia. Duran entre 3 y 4 segundos y no interrumpen lo que el usuario está haciendo. Sus momentos clave son:

- Al perder conexión: "Sin conexión — los cambios se guardarán localmente" tranquiliza al usuario de inmediato.
- Al recuperar conexión: avisa que la sincronización ya comenzó, antes de que el banner cambie de color.
- Por elemento individual: confirma sincronización exitosa en segundo plano o avisa de fallos puntuales con opción de reintentar.

Indicadores en tarjetas de plantas

Las plantas con cambios pendientes de sincronización mostrarán un ícono de nube con flecha (↑) en su tarjeta en la lista. Esto permite al usuario identificar visualmente qué datos aún no se han subido, sin necesidad de ir a otra pantalla.

Componente	Cuándo se muestra	Mensaje / Comportamiento
Banner amber	Al perder conexión	"Sin conexión · N cambios pendientes"
Banner azul	Al recuperar y sincronizar	Barra de progreso + "Sincronizando..."
Banner rojo	Si falla la sincronización	"Error al sincronizar" + botón Reintentar
Toast	Cambios de estado de red	Mensajes contextuales de 3-4 segundos
Ícono ↑ en tarjeta	Planta con cambio pendiente	Indicador visual de dato no sincronizado

3. Almacenamiento Local

3.1 Comparación de Tecnologías

Se evaluaron las principales opciones de almacenamiento local disponibles para React Native / Expo:

Criterio	AsyncStorage	expo-sqlite	MMKV	Firestore Offline
Velocidad	~70ms	Rápido con índices	~1ms (30x más rápido)	Rápido (nativo)
Datos complejos	No (solo JSON string)	Sí (tablas, queries)	No (key-value)	Sí (documentos)
Relaciones entre datos	✗	✓	✗	✓ Parcial
Soporte Expo Go	✓	✓	⚠ Dev client	⚠ Ver nota
Curva de aprendizaje	Muy baja	Media (SQL)	Baja	Baja
Integración Firebase	Manual	Manual	Manual	✓ Nativa (con RNFirebase)

⚠ **Nota importante sobre Firestore Offline Persistence en Expo:** La persistencia offline de Firestore NO funciona con el Firebase JS SDK estándar en entornos Expo / React Native. Intentar usar `enablePersistence()` o `initializeFirestore` con `persistentLocalCache()` genera el error `[code=unimplemented]: This platform is either missing IndexedDB`. Esto es un problema conocido y documentado en el repositorio oficial de `firebase-js-sdk` (issue #436, issue #7947). La única forma de tenerla es migrando a `react-native-firebase`, que requiere dev client. Fuentes: github.com/firebase/firebase-js-sdk/issues/436 | github.com/firebase/firebase-js-sdk/issues/7947

3.2 Decisión: AsyncStorage como almacenamiento principal

Dado que Firestore Offline Persistence no es compatible con el Firebase JS SDK en Expo, la estrategia de almacenamiento local para PlantasMon se basa en AsyncStorage como capa de cache y cola de sincronización manual, complementado con NetInfo para detectar el estado de red.

AsyncStorage para cache de plantas y cola de sync

- Al cargar la colección de plantas con conexión, se serializa en AsyncStorage bajo la clave `plantasmon_cache`. Si el usuario abre la app sin red, se leen los datos del cache local.
- Las operaciones de escritura (agregar, editar planta) que ocurren offline se guardan en una cola JSON en `plantasmon_offline_queue`. Al recuperar conexión, se procesan y sincronizan con Firestore.
- Este patrón está bien documentado: "En una app offline-first, los datos se almacenan localmente y en el servidor. Cuando vuelve la conexión, se sincronizan los cambios." (DEV Community, oct 2024)
- No se requieren queries complejos ni relaciones entre tablas, por lo que AsyncStorage es suficiente y evita dependencias adicionales.

Por qué no SQLite ni MMKV

- SQLite agrega complejidad de schema y migraciones innecesaria para el volumen de datos de PlantasMon (colección personal de plantas de un solo usuario).

- MMKV requiere dev client y no funciona con Expo Go, lo cual rompe el flujo de desarrollo actual del proyecto.
- AsyncStorage cubre el caso de uso: datos pequeños (JSON de plantas y cola), sin queries complejos, ya integrado en el proyecto para la sesión de auth.

3.3 Información a Guardar y Justificación

Dato	Dónde se guarda	Clave AsyncStorage	Por qué es importante
Cache colección de plantas	AsyncStorage	plantasmon_cache	Permite ver las plantas sin conexión leyendo del cache local
Cola de operaciones pendientes	AsyncStorage	plantasmon_offline_queue	Guarda ediciones/agregados offline para sincronizar al volver la red
Token de sesión	AsyncStorage	plantasmon_user	Mantiene la sesión activa entre reinicios de la app
Cola de fotos pendientes de IA	AsyncStorage	plantasmon_identify_queue	Fotos tomadas offline que esperan identificación cuando vuelva la red
Preferencias de usuario	AsyncStorage	plantasmon_prefs	Tema, idioma, configuración de notificaciones locales

3.4 Referencias

- Sanjana Human In Tech. "Approach for offline ReactNative" — Medium, oct 2023. <https://sanjanahumanintech.medium.com/approach-for-offline-reactnative-1712f1705a7b>
- msaadullah. "Building Offline-First Apps using React Native, React Query, and AsyncStorage" — DEV Community, oct 2024. <https://dev.to/msaadullah/building-offline-first-apps-using-react-native-react-query-and-asyncstorage-1h4i>
- Ramankumawat. "Mastering Offline Support in React Native" — Medium, dic 2025. <https://medium.com/@ramankumawat119/mastering-offline-support-in-react-native-build-apps-that-work-without-bd279c7f6dee>
- Oghenetega Adiri. "Building Robust Offline Functionality in React Native" — DEV Community, oct 2025. https://dev.to/oghenetega_adiri/building-robust-offline-functionality-in-react-native-a-complete-guide-4174
- SoftwareHouse.au. "Building Offline-First Mobile Apps: Design Strategies" — ago 2025. <https://softwarehouse.au/blog/building-offline-first-mobile-apps-design-strategies-for-seamless-offline-experiences/>

- Lawrence Norman. "Upgrading from Expo SDK 52 to SDK 53: Firebase JS SDK Migration" — Medium, jul 2025. <https://medium.com/@lawrencenorman7hills/upgrading-from-expo-sdk-52-to-sdk-53-firebase-js-sdk-migration-to-react-native-firebase-1c2e0ae2b544>
- firebase-js-sdk issue #436 — Firestore offline persistence not working on React Native. <https://github.com/firebase/firebase-js-sdk/issues/436>
- firebase-js-sdk issue #7947 — FR: Firestore Persistence Support for Expo / React Native. <https://github.com/firebase/firebase-js-sdk/issues/7947>
- nandorojo. expo-firestore-offline-persistence — GitHub. <https://github.com/nandorojo/expo-firestore-offline-persistence>

4. Links del Proyecto

Repositorio: <https://github.com/Z3E-Brian/Plantasmon>

Video de demostración:

https://drive.google.com/file/d/1dEsVeZflentME7QmlzsJMNhY_ltmSDgO/view?usp=sharing

API en Render: <https://plantasmon.onrender.com>

